



www.mech-lang.org

mech-lang/mech

A Rust-Native Embeddable Reactive Numerical Language for Heterogeneous Computing in Robotics

Corey Montella, Aung Kant Ko, Steven McPhillimey

Motivation

Rust's memory safety and performance make it a strong foundation for robotics. Still, numerical kernels can require substantial code for SIMD layouts and GPU execution. Backend-specific implementations can be harder to verify against the equations and maintain.

This case study covers numerical features. Mech also has first-class state machines and pattern matching. **Scan the QR code at the bottom for a demo.**

Embeddable

Mech runs standalone or embeds in Rust, from a single numerical function to a reactive subsystem. Rust hosts can compile Mech source or load a dylib. Browser applications can embed it through WebAssembly.

JIT compilation from Rust

```
use mech::kernel::{Backend, Kernel};
let source = include_str!("ekf.mec");
let kernel = Kernel::from_source(source)
    .input("bearing", [-0.55; 4])
    .export("state")
    .compile(Backend::Jit)?;
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
let state = ekf.state("state");
```

Compile once, then submit measurement updates. A failed integrity check preserves the last accepted state.

Checked dylibs[†]

Artifact	Bytes	M/s ± MAD	Peak RSS (MiB)
Mech scalar	33,544	14.671 ± 0.008	2.69 ± 0.00
Rust scalar	50,016	21.121 ± 0.005	2.69 ± 0.00
Mech SIMD-4	33,864	34.863 ± 0.010	2.69 ± 0.00

Load an AOT library

```
let kernel = unsafe { Kernel::load_bundle("ekf.bundle")? };
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
```

The bundle stores the dylib and metadata for named inputs, exported state and initial values.

Numerical

Mech offers a concrete syntax for typed matrix equations, illustrated by the bearing-only EKF below.

The Mech kernel is shorter than the textbook and SIMD Rust versions. SIMD Rust needs a separate vector and batch implementation, written directly or supplied by a library. Mech reuses the kernel across backends.

Mech compiles this syntax to a typed dataflow artifact. The interpreter uses nalgebra and could support other Rust numerical libraries. Other backends specialize the graph for the host architecture without source changes or backend-specific annotations.

Extended Kalman Filter

```
+> math/*, logic/all
Δt := 0.1
m := [140 12]'
Q := [0.01 0      -- +> import library
      0      0.0025] -- ~ mutable
R := 0.25          -- := define
I := [1 0 0        -- = assign
      0 1 0        -- ** matrix multiply
      0 0 1]       -- ' transpose
~μ := [55 25 0.4]'  -- * broadcast multiply
~Σ := [100 0 0      -- x! integrity constraint
      0 100 0
      0 0 0.15]
```

Time Update

```
θ := μ[3]
d := v * Δt
G := [1 0 (0 - d * sin(θ))
      0 1 d * cos(θ)
      0 0 1]
V := [cos(θ) * Δt 0
      sin(θ) * Δt 0
      0           Δt]
```

```
μ̄ := μ + [d * cos(θ)
          d * sin(θ)
          ω * Δt]
```

```
Σ̄ := G ** Σ ** G' + V ** Q ** V'
```

Measurement Update

```
δ := m - μ̄[1..=2]
δx := δ[1]
δy := δ[2]
q := δx ^ 2 + δy ^ 2

ẑ := atan2(δy, δx) - μ̄[3]
r := z - ẑ
v := atan2(sin(r), cos(r))

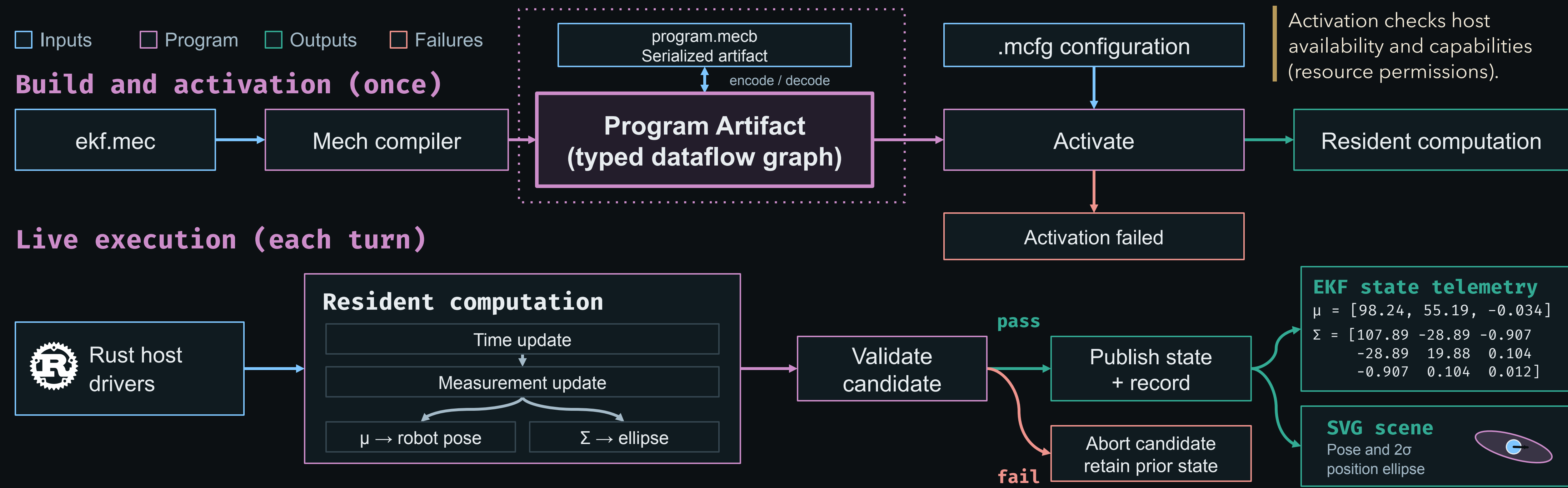
H := [δy / q, (0 - δx) / q, -1]
S := H ** Σ̄ ** H' + R
K := (Σ̄ ** H') / S[1]
A := I - K ** H

μ+ := μ̄ + K * v
Σ+ := A ** Σ̄ ** A' + (K ** K') * R

μ = μ+
Σ = Σ+
variance! := all(Σ+[[1 5 9]] > 0)
```

Reactive

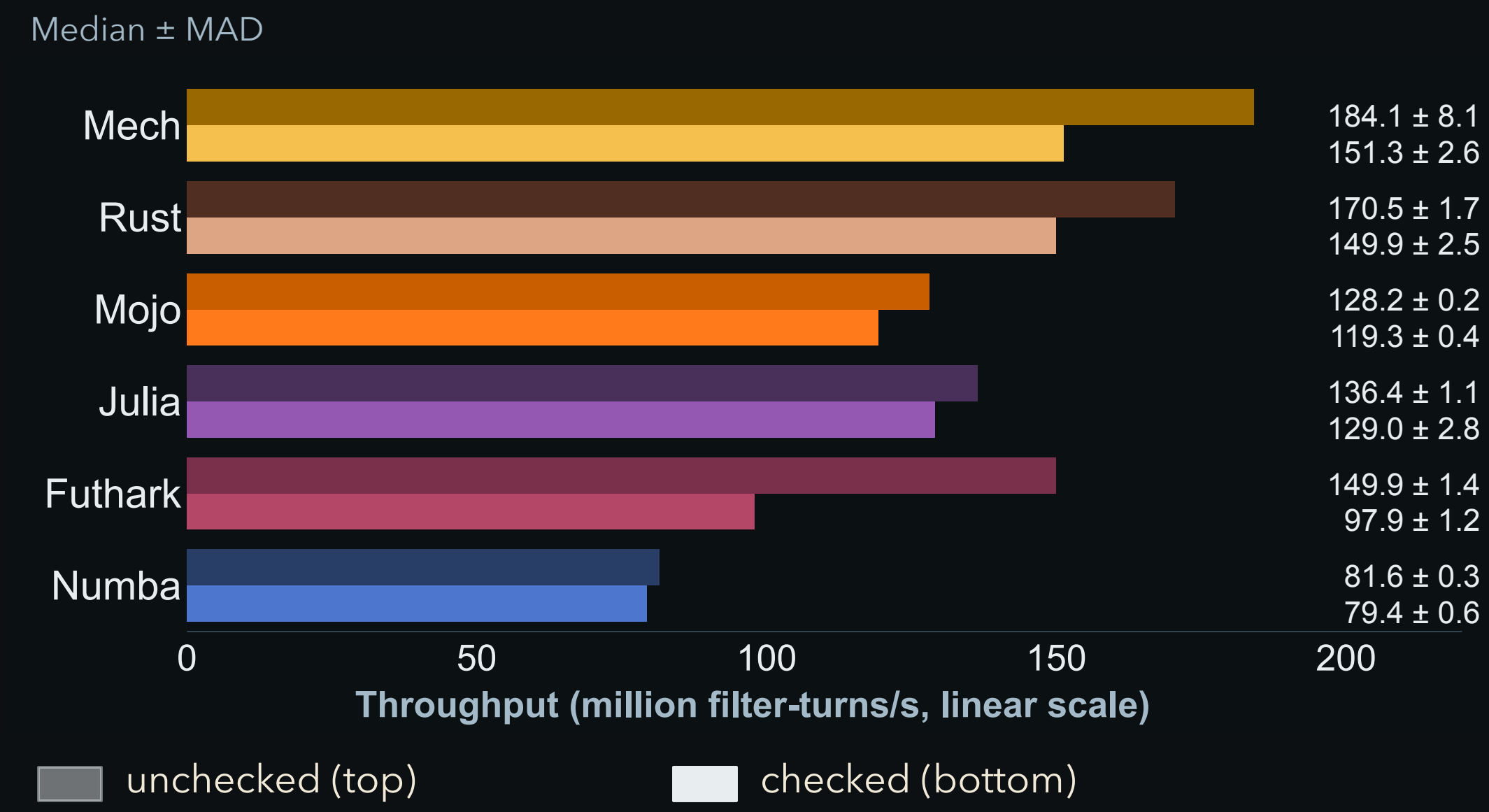
Mech models robot feedback loops over continuous sensor streams from Rust host drivers, through direct interfaces or ROS. Each update triggers a turn that computes and validates candidate state. Rejection notifies the caller and preserves prior state.



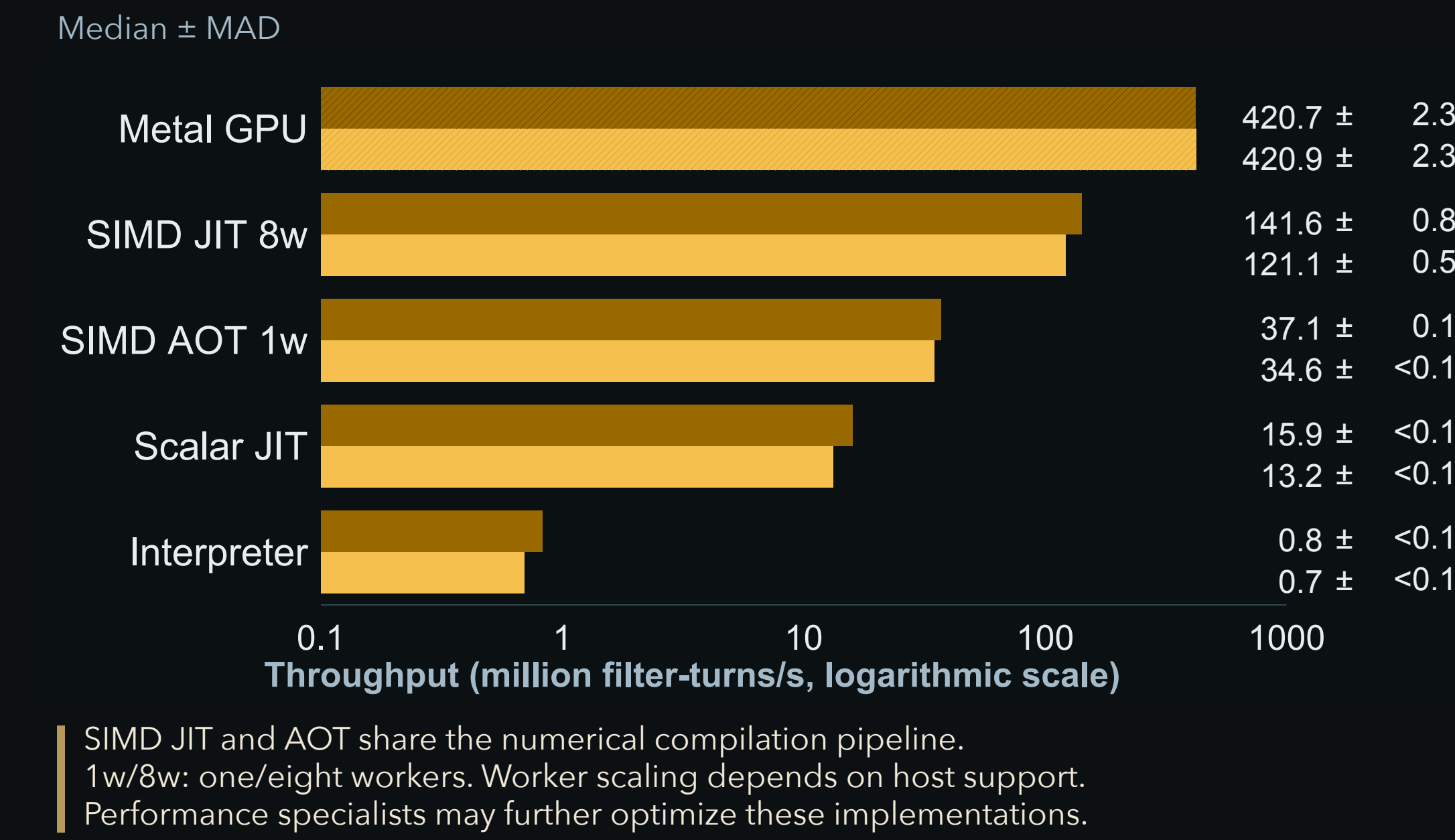
Heterogeneous

Mech's typed graph supports native CPU/GPU execution and WebAssembly/WGPU. The right chart compares five backend configurations of the displayed f32 EKF, compiled without source changes. Integrity checks add a runtime cost. Disabling them increases CPU throughput while preserving reactive turns.

Mech and Rust show similar throughput[‡]

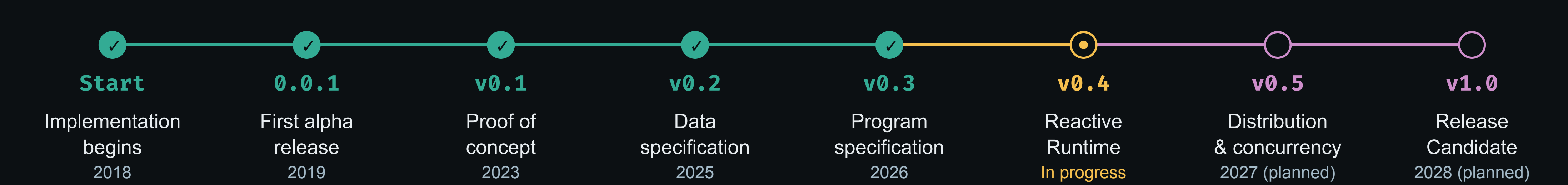


Backend choice changes EKF throughput



Status and Future Work

Mech v0.4-beta needs further validation and hardening. Over the next year, we plan to expand deployments on research robots and gather feedback. Development also includes additional hosts and language features, with a v1.0 release candidate planned for 2028.



Experience this poster as a live reactive program: CPU/WGPU in WebAssembly. Contributions and bug reports are welcome on GitHub: mech-lang/mech.



[‡] Benchmark setup. Apple M1 (4P + 4E, 8-core GPU, 8 GB), macOS 15.6.1. f32, 500,000 filters × 40 turns, n=10. CPU: 8 workers, fused turns. Mech backends: per-turn publication. Rust 1.96.0-nightly ec818fda3/LLVM 22.1.0. Cranelift 0.131.3. Julia 1.12.6. Mojo 1.1.0/MAX 26.6.0. Futhark 0.27.1/SPC 1.31.0/LLVM 22.1.8. Python 3.12.14/NumPy 2.2.6/Numba 0.61.2. Clang 17.0.0. Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c, CPU eedc1c75b, dylibs dab98af1d. iros-workshop-benchmarks, benchmarks/iros-2026. [†] Dylibs: minimal native-ABI hosts, 1 worker, 10,000 filters × 200 turns, n=7. Whole-process peak RSS. Timing excludes compilation, loading and warmup.

